

Lösungen zu den Verständnisfragen

Grundlagen

- 1.1** Ein C-Programm kann aus mehreren Quelldateien bestehen.
☒ Richtig
☐ Falsch
- 1.2** Die Übersetzung eines portablen C-Programms erzeugt eine ausführbare Datei, die auf allen Systemen lauffähig ist.
☐ Richtig
☒ Falsch (Das Programm muss für jedes System übersetzt werden.)
- 1.3** Eine Quelldatei wird mit einem Compiler übersetzt.
- 1.4** Der Linker bindet eine Objektdaten mit anderen Modulen zu einer ausführbaren Datei.
- 1.5** Standardisierte Funktionen sind in der Standardbibliothek enthalten.
- 1.6** Die Standardbibliothek ist eine Sammlung von
a) C-Quelldateien.
☒ b) Objektdateien und Header-Dateien.
c) ausführbaren Dateien.
- 1.7** Bei der Suche nach Fehlern in einem C-Programm beginnen Sie immer mit
a) dem letzten vom Compiler angezeigten Fehler.
☒ b) irgendeinem angezeigten Fehler.
c) dem ersten angezeigten Fehler.
- 1.8** Eine Warnung des Compilers kann einen
a) Syntaxfehler anzeigen.
☒ b) logischen Fehler anzeigen.
c) möglichen Laufzeitfehler anzeigen.
- 1.9** Jedes C-Programm enthält die Funktion
☒ a) main
b) scanf
c) printf
- 1.10** In einem C-Programm bedeutet das Doppelkreuz # am Anfang einer Zeile, dass diese Zeile für
a) den Compiler bestimmt ist.
☒ b) den Präprozessor bestimmt ist.
c) die Header-Datei bestimmt ist.

1.11 Die Deklarationen der Standardfunktionen für die Ein-/Ausgabe befinden sich in der Header-Datei

- ☒ a) `stdio.h`
- ☐ b) `stdlib.h`
- ☐ c) `stdint.h`

1.12 Die Programmausführung beginnt (abgesehen von allgemeinen Initialisierungen) mit

- ☐ a) der ersten `#include`-Direktive.
- ☒ b) der ersten Anweisung in der Funktion `main()`.
- ☐ c) der zuerst definierten Funktion.

1.13 Ein C-Programm gibt mit der `printf`-Funktion eine Meldung aus. Es enthält auch die Zeile

```
#include <stdio.h>
```

- ☐ a) weil jedes C-Programm diese Zeile enthalten muss.
- ☐ b) zur Deklaration der Funktion `main()`.
- ☒ c) zur Deklaration der Funktion `printf()`.

1.14 In der Funktion `main()` bewirkt die Anweisung

```
return 0;
```

- ☒ a) das Verlassen von `main()`.
- ☐ b) die Beendigung des Programms.
- ☒ c) die Rückgabe des Exitcodes 0 an das aufrufende Programm.

1.15 Die kürzeste Anweisung besteht aus

- ☒ a) einem Semikolon
- ☐ b) einer Zuweisung
- ☐ c) einer `return`-Anweisung

1.16 C-Funktionen müssen in einer bestimmten Reihenfolge definiert werden.

- ☐ Richtig
- ☒ Falsch

1.17 Die erste Funktion, die in einer Quelldatei definiert wird, ist stets die Funktion `main()`.

- ☐ Richtig
- ☒ Falsch

1.18 Der Prototyp einer Funktion liefert dem Compiler die notwendigen Informationen, um die Funktion vor ihrer Definition aufzurufen.

- ☒ Richtig
- ☐ Falsch

1.19 Zeichenfolgen werden als Kommentare interpretiert, wenn sie

- a) mit /* beginnen.
- ☒ b) in /* */ eingeschlossen sind.
- ☒ c) mit // beginnen.

1.20 In einer Zeile können mehrere Präprozessor-Direktiven stehen.

- ☐ Richtig
- ☒ Falsch

Elementare Datentypen, Konstanten und Variablen

2.1 Ein Datentyp bestimmt

- ☒ a) die Art der internen Darstellung der Daten. (welche Codierung)
- ☒ b) die Größe des benötigten Speicherplatzes.
- c) wie Daten auf dem Bildschirm angezeigt werden.

2.2 In C ist nicht festgelegt, ob der Typ char mit oder ohne Vorzeichen interpretiert wird.

- ☒ Richtig
- ☐ Falsch

2.3 Mit welchen der folgenden Datentypen können Gleitpunktzahlen dargestellt werden?

- a) long
- ☒ b) float
- ☒ c) long double

2.4 In einem portablen Programm sollen in einer Variablen ganze Zahlen im Bereich von -100000 bis 100000 gespeichert werden. In der Deklaration der Variablen ist dann der Typ long zu verwenden.

2.5 Konstanten zur Darstellung des kleinsten und größten Wertes eines ganzzahligen Typs sind definiert in der Header-Datei

- a) stdio.h
- b) math.h
- ☒ c) limits.h

2.6 Der Ausdruck

`sizeof(int)`

liefert die Größe eines Objekts vom Typ `int` in Anzahl

- a) Bits.
- ☒ b) Bytes.
- c) Megabytes.

2.7 Bei einer Genauigkeit von 6 Dezimalziffern ist garantiert, dass die Zahlen 0.0123456 und 0.0123457 unterschieden werden.

- ☒ Richtig
☐ Falsch

2.8 In einem C-Programm repräsentiert die Konstante 0xFF

- a) die Zeichen FF.
b) einen ungültigen Wert.
☒ c) den dezimalen Wert 255.

2.9 Welche der folgenden Konstanten haben einen Gleitpunkttyp?

- ☒ a) 7.
☐ b) 70
☒ c) 7E-1

2.10 Welche numerischen Werte repräsentieren folgende Konstanten?

- a) 0 0.
b) '0' 48. (ASCII von 0)
c) '\0' 0. (Spezialzeichen mit dem Wert 0; ist 1 Byte gross; "terminierende 0" für Zeichenketten, um das Ende zu signalisieren)

2.11 Die Konstanten 'A' und "A" sind gleichwertig. Beide repräsentieren das Zeichen A.

- ☐ Richtig
☒ Falsch ('A' ist der Zeichencode von A, also 65, und "A" ist ein String.)

2.12 Der String "Hi " belegt 3 Bytes. (ein Byte für das String-Endezeichen \0)

2.13 Die Escape-Sequenzen \t und \n repräsentieren die Steuerzeichen Horizontaler Tabulator und Zeilenwechsel (new-line).

2.14 Stringkonstanten, die nur durch Zwischenraumzeichen (Blanks, Tabs und Newline-Zeichen) getrennt sind, werden zu einem String zusammengezogen.

- ☒ Richtig
☐ Falsch

2.15 Bei welcher der Zeichenfolgen handelt es sich um einen gültigen Namen?

- ☒ a) _A_
b) Hans-Otto (" – " kann als Operator interpretiert werden)
c) 1x (darf nicht mit numerischen Zeichen beginnen)

2.16 Welche der folgenden Deklarationen sind korrekt?

- a) unsigned int n = -100;
☒ b) char c = '\t';
☒ c) float x = -12.345F

2.17 Bei der Definition einer lokalen Variablen ohne Initialisierung wird

- ☒ a) der Typ und Name der Variablen festgelegt.
☒ b) der Speicherplatz für die Variable reserviert.
c) der Anfangswert 0 der Variablen automatisch zugewiesen.

2.18 Jede globale Variable ohne explizite Initialisierung wird mit 0 vorbelegt.

2.19 Eine ganze Zahl vom Typ `double` soll mit der Funktion `printf()` dezimal angezeigt werden. Welches Formatelement ist richtig?

a) `%C`

b) `%d`

c) `%f`

2.20 Das Datenobjekt `limit` ist wie folgt definiert:

```
const int limit = 100;
```

Welche Anweisungen sind zulässig?

a) `int start = limit / 2;`

b) `printf("Grenzwert: %d\n", limit);`

c) `limit = 200;`

Verwendung von Funktionen

3.1 In C muss jede Funktion vor ihrem Aufruf deklariert werden.

3.2 In C sind die beiden folgenden Deklarationen gleichwertig:

```
double func();  
double func(void);
```

☐ Richtig

☒ Falsch (leere Klammern bedeuten keine Information über Parameter)

3.3 Der Prototyp einer Funktion informiert den Compiler über

a) die Anzahl der Parameter.

b) den Typ jedes Parameters.

c) den Typ des Return-Wertes.

3.4 Eine Funktion darf auch mehrmals deklariert werden:

☒ Richtig

☐ Falsch

3.5 Der Aufruf einer Funktion ist stets ein Ausdruck vom Typ

a) `void`.

b) der im Funktionskopf deklarierten Parameter.

c) des Return-Wertes der Funktion oder `void`.

3.6 Parameternamen im Prototyp einer Funktion haben nur die Bedeutung eines Kommentars und können auch weggelassen werden.

☒ Richtig

☐ Falsch

3.7 Für den Aufruf einer Standardfunktion benötigt der Compiler

- a) keine weiteren Informationen.
- b)** die Deklaration der Funktion.
- c) die Definition der Funktion.

3.8 Ein Argument, das einer Funktion übergeben wird, darf ein beliebiger Ausdruck vom Typ des entsprechenden Parameters sein.

3.9 Die Funktion `myFunc()` besitzt folgenden Prototyp

```
long myFunc( double, int);
```

Dann hat der Funktionsaufruf

```
myFunc( 31.7, 5)
```

den Typ

- a)** long
- b) void
- c) int

3.10 In C gibt jede Funktion einen Wert zurück.

- ☐ Richtig
- ☒ Falsch (Funktionen, die keinen Wert zurückgeben, haben den Typ void.)

3.11 Zur Deklaration einer Standardfunktion für die Ein- oder Ausgabe, beispielsweise der Funktion `printf()`, wird die Header-Datei stdio.h inkludiert.

3.12 Eine Header-Datei ist eine

- a)** Textdatei.
- b) Objektdati.
- c) ausführbare Datei.

3.13 Alle benötigten Header-Dateien stellt die Compiler-Software zur Verfügung. Es ist daher nie notwendig, eigene Header-Dateien zu schreiben.

- ☐ Richtig
- ☒ Falsch

3.14 In einem Programm wird die Header-Datei `math.h` inkludiert und die Standardfunktion `pow()` mit nur einem Argument aufgerufen. Der Compiler

- a) verwendet für den zweiten Parameter einen Default-Wert.
- b) gibt eine Warnung aus.
- c)** gibt eine Fehlermeldung aus.

3.15 Die Standardfunktion, die eine Zufallszahl zurückgibt, heißt

- a)** `rand()`
- b) `random()`
- c) `srand()`

3.16 Zur Deklaration der Standardfunktionen `srand()` und `rand()` ist folgende Header-Datei zu inkludieren:

- a) `stdio.h`
- b) `stdlib.h`**
- c) `math.h`

3.17 Ein C-Programm definiert die zwei `int`-Variablen `zahl1` und `zahl2` und enthält die Anweisungen:

```
zahl1 = rand();  
zahl2 = rand();
```

Anschließend enthalten die Variablen `zahl1` und `zahl2` denselben Wert, da zweimal dieselbe Funktion ohne ein Argument aufgerufen wird.

☐ Richtig

☒ Falsch (`rand()` liefert mit jedem neuen Aufruf eine neue Zufallszahl.)

3.18 Damit der Compiler die Header-Datei `myProject.h` zuerst im aktuellen Verzeichnis sucht, muss die `include`-Direktive wie folgt lauten:

- a) `#include "myProject.h"`**
- b) `#include <myProject.h>`
- c) `#include myProject.h`

3.19 Ein C-Programm ruft die Standardfunktion `sqrt()` auf, um die Quadratwurzel einer Zahl zu berechnen. Dazu inkludiert das Programm die Header-Datei

- a) `math.h`**
- b) `stdlib.h`
- c) `stdio.h`

3.20 Ein C-Programm definiert folgende Variablen:

```
double x = 2.0, y;
```

Die Anweisung, um mit der Standardfunktion `sqrt()` aus `x` die Quadratwurzel zu ziehen und in `y` zu speichern, lautet dann:

- a) `y = sqrt(x);`**
- b) `y = sqrt().x;`
- c) `sqrt(x, y);`

Formatierte Ein- und Ausgabe

4.1 Die Funktionen für die formatierte Ein- und Ausgabe sind in der Header-Datei `stdio.h` deklariert.

4.2 `printf()` ist eine Funktion, die mit einer unterschiedlichen Anzahl von Argumenten aufgerufen werden kann. Das erste Argument ist stets ein String, der für jedes weitere Argument ein Formatelement (= Konvertierungsspezifikation) enthält.

4.3 Um den Wert *einer* Variablen auszugeben, wird `printf()` mit 2 Argumenten aufgerufen.

4.4 In einem C-Programm, das mit einer `double`-Variable `x` arbeitet, bewirkt die Anweisung:

```
printf("Wert von x = %d\n", x);
```

- a) die korrekte dezimale Ausgabe des Wertes von `x`.
- ☒ b) eine unsinnige Ausgabe.
- c) eine Fehlermeldung des Compilers.

4.5 Die `printf`-Anweisung, um den Wert einer Variablen `n` vom Typ `unsigned int` dezimal, oktal und hexadezimal anzuzeigen, lautet:

- ☒ a) `printf("%u %o %x", n, n, n);`
- b) `printf("%d %o %x", n, n, n);`
- c) `printf("%i %o %x", n, n, n);`

4.6 Bei der Ausgabe von Ganzzahlen im oktalen oder hexadezimalen Format werden diese stets ohne Vorzeichen interpretiert.

- ☒ [X] Richtig
- ☐ [] Falsch

4.7 Die Variablen `temp1` und `temp2` vom Typ `float` enthalten zwei gemessene Temperaturen. Dann gibt die Anweisung

```
printf("Die Temperaturen: %.1f\n", temp1, temp2);
```

- ☒ a) nur den Wert von `temp1` aus. Das 3. Argument `temp2` wird ignoriert.
- b) beide Temperaturen im gleichen Format aus.
- c) einen unsinnigen Wert aus.

4.8 Das Formatelement zur dezimalen Ausgabe eines Wertes vom Typ `long` lautet

- ☒ a) `%ld`
- b) `%lf`
- c) `%lu`

4.9 Der Standardwert für die Genauigkeit bei der Ausgabe von Gleitpunktzahlen ist

- a) 2
- ☒ b) 6
- c) 10

4.10 Um zu erreichen, dass die Ausgabe in einem Feld linksbündig erfolgt, wird der Feldbreite ein Minuszeichen vorangestellt.

- ☒ [X] Richtig
- ☐ [] Falsch

4.11 Wenn die auszugebende Zeichenfolge länger als die angegebene Feldbreite ist, wird die Ausgabe abgeschnitten.

- ☐ [] Richtig
- ☒ [X] Falsch

4.12 Ist die Feldbreite größer als die auszugebende Zeichenfolge, werden restliche Stellen im Feld standardmäßig mit folgendem Zeichen aufgefüllt:

- a) Tabulator
- b) Punkt
- ☒ c) Blank

4.13 Eine Gleitpunktzahl vom Typ `double` soll rechtsbündig in einem Feld der Breite 12 mit zwei Stellen nach dem Dezimalpunkt angezeigt werden. Das Formatelement dafür lautet

- ☒ a) `%12.2f` oder `%12.2lf`
- b) `%12.2x` oder `%12.2lu`
- c) `%12.2i` oder `%12.2ld`

4.14 Den Wert, den die Funktion `scanf()` eingelesen hat, liefert die Funktion als Return-Wert.

- ☐ Richtig
- ☒ Falsch

4.15 Um mit `scanf()` oder `scanf_s()` einen Wert in eine Variable einzulesen, muss als weiteres Argument die Adresse der Variablen angegeben werden.

4.16 Die Anweisung, um mit der Funktion `scanf()` eine Zahl in die `double`-Variable `var` einzulesen, lautet

- ☒ a) `scanf("%lf", &var);`
- b) `scanf("%lf", var);`
- c) `var = scanf("%f");`

4.17 Ein C-Programm liest mit einer `scanf`-Anweisung das Alter des Anwenders in eine Variable vom Typ `int` ein. Der Anwender gibt aber seinen Namen ein. Dieser Fehler wird angezeigt durch

- ☒ a) den Return-Wert von `scanf()`.
- b) den Wert 0 in der `int`-Variablen.
- c) eine automatische Fehlermeldung.

4.18 Beim Einlesen von Zahlen mit der Funktion `scanf()` oder `scanf_s()` werden führende Zwischenraumzeichen ignoriert.

- ☒ Richtig
- ☐ Falsch

4.19 Was bewirkt die Angabe der Feldbreite in folgender `scanf`-Anweisung:

```
int n; scanf("%5d", &n);
```

- a) Nichts. Feldbreiten haben bei `scanf()` keine Bedeutung.
- b) Es müssen mindestens 5 Ziffern eingegeben werden.
- ☒ c) `scanf()` liest maximal 5 Ziffern von der Standardeingabe.

- 4.20** In die zwei float-Variablen t1 und t2 sollen mit folgender Anweisung zwei gemessene Temperaturen eingelesen werden:

```
int ret = scanf("%f %f", &t1, &t2);
```

Nach der Eingabe von 9.3 und 23.7 haben die Variablen folgende Werte:

ret = 2 , t1 = 9.3 und t2 = 23.7 .

Operatoren

- 5.1** In C hat jeder Ausdruck mit Ausnahme der Zuweisung einen Wert.

☐ Richtig

☒ Falsch

- 5.2** Das Ergebnis der Division 3 / 2 ist 1 und hat den Typ int .

- 5.3** Der Ausdruck 7 - 3 * 3 + 2 hat den Wert

☒ a) 0.

☐ b) 14.

☐ c) 20.

- 5.4** In Ausdrücken kann die Zuordnung der Operanden zu den Operatoren durch Setzen von Klammern geändert werden.

- 5.5** Die Operanden der Modulo-Division müssen einen ganzzahligen Typ haben.

☒ Richtig

☐ Falsch

- 5.6** Der Ausdruck 2 * 3 % 4 hat den Wert 2 .

- 5.7** Welche der folgenden Ausdrücke haben den Wert 0?

☒ a) -3 * 2 + 13 % 7

☒ b) 13 / 2 - 2 * 3

☒ c) 8 % 3 - 8 / 3

- 5.8** Die Operatoren ++ und -- haben den gleichen Vorrang wie die Operatoren + und - .

☐ Richtig

☒ Falsch

- 5.9** Nach Ausführung des Code-Fragments

```
int a = 1, b = 1;  
int c = a++ + ++b;
```

hat die Variable c den folgenden Wert:

☐ a) 2

☒ b) 3

☐ c) 4

5.10 Der Ausdruck `a++ * b-- % 2` mit den arithmetischen Variablen `a` und `b` ist äquivalent zum Ausdruck `(a++) * ((b--) % 2)`.

☐ Richtig

☒ Falsch Richtig ist `((a++)*(b--))%2`

5.11 Ein C-Programm enthält die Definition:

```
const int max = 100;
```

Dann bewirkt die Anweisung:

```
++max;
```

☒ a) eine Fehlermeldung des Compilers.

☐ b) die Inkrementierung von `max`.

☐ c) nichts, da `max` konstant ist.

5.12 Die Operatoren `*=` und `/=` haben den gleichen Vorrang wie die einfache Zuweisung `=`.

☒ Richtig

☐ Falsch

5.13 Nach Ausführung des Code-Fragments

```
int var = 6;  
var /= 2 + 1;
```

hat die Variable `var` den Wert 2.

5.14 Mit der Definition

```
float x = 2.0F, y = 0.0F;
```

bewirkt die Anweisung:

```
x *= y = 5.0F;
```

☐ a) eine Fehlermeldung des Compilers.

☐ b) die Zuweisung von `5.0` an die Variablen `x` und `y`.

☒ c) die Zuweisung von `5.0` an `y` und die Zuweisung von `10.0` an `x`.

5.15 In C ist ein Vergleich ein Ausdruck vom Typ

☒ a) `int`

☐ b) `bool`

☐ c) `char`

5.16 Das Code-Fragment

```
int flag, i = 1;  
printf("%d", flag = 1 == 2);
```

erzeugt folgende Ausgabe:

- ☒ a) 0
- ☐ b) 1
- ☐ c) 2

5.17 Die Variablen a und b im Ausdruck

```
10 * a++ + b-- / 2 > 0
```

haben einen arithmetischen Typ. Die vollständige Klammerung des Ausdrucks gemäß den Vorrangregeln ergibt den äquivalenten Ausdruck

- ☒ a) $((10 * (a++)) + ((b--)/2)) > 0$
- ☐ b) $((10 * a)++) + ((b--)/2)) > 0$
- ☐ c) $((10 * (a++)) + ((b--)/2)) > 0$

5.18 Jeder Operand eines booleschen Operators wird als wahr interpretiert, wenn sein Wert ungleich null ist.

- ☒ Richtig
- ☐ Falsch

5.19 Angenommen, die int-Variable a hat den Wert 0 und die int-Variable b den Wert 1. Dann hat die Bedingung

```
a < 10 || a > 20 && b == 0
```

den Wert 1, das heißt, die Bedingung ist wahr.

5.20 Ein C-Programm enthält die Definition:

```
int result, n = -1;
```

Dann bewirkt die Anweisung:

```
result = n++ < 0 && ++n > 0;
```

- ☐ a) die Zuweisung von 0 an result.
- ☒ b) die Zuweisung von 1 an result.
- ☐ c) eine Fehlermeldung des Compilers, da die Variable n in einem Ausdruck zweimal verändert wird.

Kontrollstrukturen

- 6.1** Die Schleifenanweisungen in C verwenden zur Kontrolle der Schleife eine Laufbedingung.

☒ Richtig

☐ Falsch

- 6.2** In einer for-Schleife soll die `int`-Variable `n` die geraden Zahlen von 2 bis einschließlich 20 durchlaufen. Der Schleifenkopf dafür lautet

```
for( n = 2; n <= 20; n += 2)
```

☒ Richtig

☐ Falsch

- 6.3** Der Schleifenkopf `for ( ; ; )`

a) ist unzulässig.

☒ **b)** ist äquivalent zu `while (1)`.

☒ **c)** erzeugt eine Endlosschleife.

- 6.4** Eine Schleife kann jederzeit mit der Anweisung `break` beendet werden.

- 6.5** Die Anweisungen einer Schleife werden mindestens einmal ausgeführt.

☐ Richtig

☒ Falsch

- 6.6** Gegeben ist folgende Schleife:

```
int i, sum;
for( sum = 0, i = 1; i < 5; ++i)
    sum += i;
```

Nach Ausführung der Schleife enthält `sum` den Wert 10.

- 6.7** Die for-Schleife aus Frage 6.6 ist äquivalent mit:

a)

```
sum = 0; i = 0;
while( i++ < 5 ) sum += i;
```

b)

```
sum = 0; i = 1;
while( ++i < 5 ) sum += i;
```

☒ **c)**

```
sum = 0; i = 1;
do sum += i; while( ++i < 5 );
```

6.8 Die folgende Schleife ist eine Endlosschleife.

```
int i = 10;
while( i > 0)
    printf("%d ", i); --i;
```

☒ Richtig

☐ Falsch

6.9 Die Schleife

```
int a = 0, b = 10;
for(;;) {
    printf("%d\n", b - a);
    if( ++a >= --b) break; }
```

kann übersichtlicher ohne if-Anweisung als do-while-Schleife geschrieben werden.
Diese lautet:

```
do printf("%d\n", b - a); while( ++a < --b);
```

☒ Richtig

☐ Falsch

6.10 Nach Ausführung der folgenden Schleife

```
int z = 0, erg = 10;
do { erg += erg; ++z; } while( erg < 100);
```

haben die Variablen z und erg folgende Werte:

a) z == 3 und erg == 80

b) z == 3 und erg == 160

☒ **c)** z == 4 und erg == 160

6.11 Nach Ausführung des Code-Fragments

```
int i = 0, var = 100;
if( i != 0) var *= 2;
else var /= 2;
```

hat die Variable var den Wert 50.

6.12 Die if-else-Anweisung aus Frage 6.11 ist äquivalent mit

```
switch(i)
{ case 0: var /= 2; break;
default: var *= 2;
}
```

☒ Richtig

☐ Falsch

6.13 Nach Ausführung der Anweisungen

```
int a = 10, b = 0;
if( a > 10)
    if( a <= 20) b = 1;
    else b = 2;
```

hat die Variable b den Wert 0.

6.14 Nach Ausführung der Anweisungen

```
int a = 10, b = 0;
if( a > 10)
{ if( a <= 20) b = 1; }
else b = 2;
```

hat die Variable b den Wert 2.

6.15 Welche Ausgabe erzeugt die folgende `else-if`-Kette?

```
double speed = 78.5, amount = 200.0;
if( speed > 100.0)      printf("%.2f\n", amount);
else if( speed > 80.0)  printf("%.2f\n", amount/2);
else if( speed > 60.0)  printf("%.2f\n", amount/4);
```

- ☒ a) 50.00
- ☐ b) 100.00
- ☐ c) 200.00

6.16 Die `else-if`-Kette aus Frage 6.15 kann durch eine äquivalente `switch`-Anweisung ersetzt werden.

- ☐ Richtig
- ☒ Falsch

6.17 In einer `switch`-Anweisung

```
switch( expression) { . . . }
```

muss der Ausdruck `expression` folgenden Typ haben:

- ☐ a) einen numerischen Typ
- ☒ b) einen ganzzahligen Typ
- ☐ c) den Typ `int`

6.18 Für eine `double`-Variable `x` mit dem Wert `-10.5` wird durch

```
int s = x > 0.0 ? 1 : (x < 0.0 ? -1 : 0);
```

der Wert -1 an die Variable `s` zugewiesen.

6.19 Die Anweisung `goto` ermöglicht es, zu einer beliebigen Anweisung im selben Programm zu springen, wenn die Anweisung durch eine Marke gekennzeichnet ist.

☐ Richtig

☒ Falsch

6.20 Die ersten drei Zahlen und die letzten drei Zahlen, die von der Schleife

```
for( int j = 1900; j < 2010; j += 4) {  
    if( j%100 == 0 && j%400 != 0) continue;  
    printf("%6d", j);  
}
```

ausgegeben werden, sind:

☒ a) 1904 1908 1912 ... 2000 2004 2008

☐ b) 1900 1904 1908 ... 2000 2004 2008

☐ c) 1904 1908 1912 ... 2001 2002 2003

Symbolische Konstanten und Makros

7.1 Die Definition eines Makros erfolgt mit der `#define`-Direktive des Präprozessors.

7.2 Bei welchen Direktiven handelt es sich um zulässige Makrodefinitionen?

☒ a) `define LIMIT 1000`

☐ b) `#undef LIMIT`

☐ c) `Version: #define VERSION 1.52`

☒ d) `#define SIZE (16*512)`

☐ e) `#define MB-RAM 4096`

7.3 Die Verwendung bereits definierter Makros in neuen Makrodefinitionen ist zulässig.

☒ Richtig

☐ Falsch

7.4 Das Code-Fragment

```
#define MB_RAM 4096  
printf("Arbeitsspeicher(MB): MB_RAM ");
```

erzeugt die Ausgabe: Arbeitsspeicher(MB): MB_RAM.

7.5 Die Makros

```
#define VAT_RATE 19.0  
#define VAT(amount) (amount*VAT_RATE/100)
```

werden in der Anweisung

```
tax = VAT(11.50 + 6.10)
```

wie folgt erweitert:

- a) `(11.50 + 6.10*VAT_RATE/100)`
- ☒ b) `(11.50 + 6.10*19.0/100)`
- c) `(11.50 + 6.10)*19.0/100`

7.6 Das Makro VAT aus der Frage 4.5 liefert bei einem Aufruf wie `VAT(11.50+6.10)` ein falsches Ergebnis. Eine korrekte Definition von VAT ist:

- ☒ a) `#define VAT(amount)((amount)*VAT_RATE/100.0)`
- b) `#define VAT(amount)((amount * VAT_RATE)/100.0)`
- c) `#define VAT(amount)((x) * VAT_RATE / 100)`

7.7 Das Makro

```
#define MAX 100;
```

kann mit einer `int`-Variablen `n` wie folgt verwendet werden:

```
if(n > MAX) printf("Wert von n ist zu gross!\n");
```

- ☐ Richtig
- ☒ Falsch (Das Semikolon nach 100 ist zu viel.)

7.8 Bei welchen der folgenden Definitionen handelt es sich um ein Makro mit einem Parameter?

- ☒ a) `#define SquareOf(x) ((x)*(x))`
- b) `#define SQUAREOF x ((x)*(x))`
- c) `#define SquareOf (x) (x)*(x)`

7.9 Um eine Makrodefinition in einer neuen Zeile fortzusetzen, muss die aktuelle Zeile mit dem Zeichen `\` abgeschlossen werden.

7.10 Das Makro ABS ist wie folgt definiert:

```
#define ABS(x) ((x)>=0 ? (x) : -(x))
```

Für eine `int`-Variable `a` mit dem Wert `-2` liefert der Aufruf `ABS(++a)` den Wert `0`.

- ☒ Richtig
- ☐ Falsch

- 7.11** Makros, die in einer Ihrer Quelldateien definiert sind, wollen Sie auch in anderen Quelldateien verwenden. Zu diesem Zweck ist es sinnvoll,
- a) die Quelldatei mit den Makrodefinitionen in den anderen Quelldateien zu inkludieren.
 - ☒ b) eine Header-Datei mit den Makrodefinitionen zu erstellen und diese in den Quelldateien zu inkludieren.
 - c) die Makrodefinitionen in jede Quelldatei zu kopieren.
- 7.12** Um einem Makro eine andere Bedeutung zu geben, genügt es, das Makro mit gleichem Namen erneut zu definieren.
- ☐ Richtig
 - ☒ Falsch
- 7.13** Um Standardmakros wie `isdigit()` oder `toupper()` aufrufen zu können, muss die Header-Datei `ctype.h` inkludiert werden.
- 7.14** Die ausführbare Datei eines C-Programms enthält den Maschinencode eines Makros
- ☒ a) so oft, wie das Makro im Quellcode aufgerufen wird.
 - b) genau einmal.
 - c) keinmal.
- 7.15** Die Definition eines Makros kann mit der Direktive `#undef` aufgehoben werden.
- 7.16** Beim Aufruf eines Funktionsmakros überprüft der Compiler die aktuellen Argumente wie bei einem gewöhnlichen Funktionsaufruf.
- ☐ Richtig
 - ☒ Falsch
- 7.17** Das Code-Fragment

```
#ifndef MyMakros
#define MyMakros
...
#endif
```

bewirkt, dass der Quellcode im Block zwischen `#ifndef` und `#endif` nicht kompiliert wird, wenn das Makro `MyMakros` bereits definiert ist.

- ☒ Richtig
- ☐ Falsch

7.18 Ein Quellcode beginnt mit folgenden Präprozessor-Direktiven:

```
#if defined(__STDC_LIB_EXT1__) || defined(_MSC_VER)
    #define __STDC_WANT_LIB_EXT1__ 1
    #define SCANF scanf_s
#else
    #define SCANF scanf
#endif
```

Wenn dann eines der Makros `__STDC_LIB_EXT1__` oder `_MSC_VER` definiert ist, ergibt im nachfolgenden Quellcode die Erweiterung von `SCANF` die Zeichenfolge `scanf_s`.

Hinweis: Für Informationen zur Funktion `scanf_s()` siehe Kapitel 4. Das Makro `_MSC_VER` ist definiert, wenn die Quelldatei von einem Microsoft-Compiler übersetzt wird.

7.19 Das Filterprogramm

```
#include <stdio.h>
#include <ctype.h>
int main()
{ int c;
  while( (c = getchar()) != EOF)
  { c = toupper(c); putchar(c); }
  return 0;
}
```

- a) filtert alle Zeichen aus, die keine Großbuchstaben sind.
- b) filtert alle Zeichen aus, die keine Buchstaben sind, und wandelt Klein- in Großbuchstaben um.
- ☒ c) filtert keine Zeichen aus, wandelt aber Klein- in Großbuchstaben um.

7.20 Angenommen, `zeilen.exe` ist ein ausführbares Filterprogramm, das einen Text mit Zeilennummern versieht, und Ihr Betriebssystem ermöglicht die Umlenkung der Ein- und Ausgabe. Dann lautet die Kommandozeile, um im aktuellen Verzeichnis die Textdatei `file1.txt` mit `zeilen.exe` zu lesen und das Ergebnis als `file2.txt` zu speichern:

```
zeilen <file1.txt >file2.txt
```

- ☒ Richtig
- ☐ Falsch

Typumwandlungen

8.1 Eine arithmetische Operation wie z.B. die Addition kann mit Operanden verschiedenen Typs durchgeführt werden, ohne dass der Compiler eine Typumwandlung vornehmen muss.

- ☐ Richtig
- ☒ Falsch

- 8.2** Wenn die Operanden eines binären arithmetischen Operators einen unterschiedlichen arithmetischen Typ haben, führt der Compiler die üblichen Arithmetischen Typumwandlungen durch.
- 8.3** In C wird mindestens mit Operanden vom Typ `int` gerechnet, das heißt, für Operanden mit einem kleineren Typ wird die Ganzzahlerweiterung durchgeführt.
- 8.4** Die üblichen arithmetischen Typumwandlungen werden angewendet auf die arithmetischen Operanden
- a) aller arithmetischen Operatoren.
 - b) aller binären Operatoren.
 - ☒ c) aller binären arithmetischen Operatoren und Vergleiche.
- 8.5** Die üblichen arithmetischen Typumwandlungen erfolgen auf der Basis der Typhierarchie. Generell gilt:
- a) Der »größere« Typ wird in den »kleineren« Typ umgewandelt.
 - ☒ b) Der »kleinere« Typ wird in den »größeren« Typ umgewandelt.
 - c) Die Umwandlung hängt vom beteiligten Operator ab.
- 8.6** Die üblichen arithmetischen Typumwandlungen sind nicht immer werterhaltend, das heißt, der Wert eines Operanden kann sich bei der Typumwandlung verändern.
- ☒ Richtig
☐ Falsch
- 8.7** Wenn auf einem System der Typ `int` eine größere Breite (= Anzahl Bits) als der Typ `short` hat, wird bei der Ganzzahlerweiterung der Typ `unsigned short` umgewandelt in
- a) `unsigned short` (unverändert).
 - ☒ b) `int`.
 - c) `unsigned int`.
- 8.8** Bei der Umwandlung eines Wertes vom Typ `long` in den Typ `double` bleibt das Bitmuster erhalten.
- ☐ Richtig
☒ Falsch
- 8.9** Bei der Umwandlung eines negativen Wertes vom Typ `int` in den Typ `unsigned int`
- ☒ a) bleibt das Bitmuster erhalten, wird aber ohne Vorzeichen interpretiert.
 - b) wird das Einer-Komplement gebildet.
 - c) wird das Zweier-Komplement gebildet.

8.10 Gegeben sei die Definition:

```
long n = -1L;
```

Dann hat der Ausdruck `(n < 0 ? 0 : 7.5)` den Typ

- ☒ a) `double`
- b) `int`
- c) `unsigned int`

- 8.11** Angenommen, in einem Ausdruck muss ein Wert vom Typ `short` in den Typ `long` konvertiert werden. Dies erfolgt mit
- a) der Null-Erweiterung, das heißt, die höherwertigen Bits werden mit 0 aufgefüllt.
 - ☒ b) der Vorzeichen-Erweiterung, das heißt, die höherwertigen Bits werden mit dem Vorzeichenbit des Wertes aufgefüllt.
 - c) keiner der zwei genannten Erweiterungen.

8.12 Das Code-Fragment

```
int i = -1; unsigned int limit = 10;
if( i < limit) printf("Im if-Zweig.\n");
else          printf("Im else-Zweig.\n");
```

erzeugt die Ausgabe:

- a) Im if-Zweig.
- ☒ b) Im else-Zweig.

8.13 In einer Zuweisung mit arithmetischen Typen

- a) werden die üblichen arithmetischen Typumwandlungen durchgeführt.
- b) wird die Variable auf der linken Seite in den Typ des Wertes auf der rechten Seite umgewandelt.
- ☒ c) wird der Wert auf der rechten Seite in den Typ der Variablen auf der linken Seite umgewandelt.

8.14 Nach Ausführung des folgenden Code-Fragments

```
int a = 5, b = 10; double quotient;
quotient = a/b;
```

hat die Variable `quotient` den Wert 0.0.

8.15 Bei einer Zuweisung eines Wertes vom Typ `long` an eine `float`-Variable kann es zu Rundungsfehlern kommen.

- ☒ Richtig
- ☐ Falsch

8.16 Der Compiler gibt gewöhnlich eine Warnung aus, wenn es bei einer *impliziten* Typumwandlung zu Datenverlust kommen kann. Um eine solche Warnung zu vermeiden, kann der Programmierer eine Typumwandlung auch explizit durchführen.

8.17 Gegeben sei eine Variable `n` vom Typ `int`. Die explizite Typumwandlung

`(double)n`

vergrößert die Variable `n` so, dass sie `double`-Werte speichern kann.

- ☐ Richtig
- ☒ Falsch

8.18 Nach folgenden Anweisungen

```
long g; double x = 3 * 3.3;  
g = (long)x;
```

hat die Variable g den Wert 9.

8.19 Mit zwei int-Variablen i und j wird der Ausdruck

```
(double)i*j;
```

wie folgt ausgewertet:

- a) Das Ergebnis der Multiplikation `i*j` wird explizit in `double` umgewandelt.
- b) Die Variablen `i` und `j` werden explizit in `double` umgewandelt und anschließend multipliziert.
- ☒ c) Zuerst wird die Variable `i` explizit in `double` umgewandelt. Dann wird `j` implizit in `double` umgewandelt, um anschließend die Multiplikation auszuführen.

8.20 Gegeben seien folgende Deklarationen:

```
void calculate( double, long);  
int n = 3; double x = 1.7;
```

Dann werden der Funktion beim Aufruf

```
calculate(n/2, x+0.5); // evtl. eine Warnung des Compilers.
```

folgende zwei Werte übergeben:

- a) 1.0 und 1
- ☒ b) 1.0 und 2
- c) 1.5 und 2

Vektoren und Strings

9.1 Vektoren können nicht lokal innerhalb eines Blocks definiert werden.

- ☐ Richtig
- ☒ Falsch

9.2 In C hat das erste Element eines Vektors stets den Index

- ☒ a) 0
- b) NULL
- c) 1
- d) i

9.3 Die Elemente eines Vektors belegen einen zusammenhängenden Speicherbereich.

- ☒ Richtig
- ☐ Falsch

9.4 Für einen beliebigen Vektor `arr` liefert der Ausdruck

`sizeof(arr)/sizeof(arr[0])`

- a) die Größe des von `arr` belegten Speichers in Anzahl Bytes.
- ☒ b) die Länge des Vektors `arr`.
- c) die Anzahl Bytes pro Vektorelement.

9.5 Gegeben sei die symbolische Konstante `LEN`, die eine positive Ganzzahl repräsentiert, und die Vektordefinition

```
int v[LEN];
```

Dann wird dem letzten Vektorelement mit folgender Anweisung der Wert 0 zugewiesen:

- ☒ a) `v[LEN - 1] = 0;`
- b) `v[LEN] = 0;`
- c) `v[i] = 0;`
- d) `v[strlen(v)] = 0;`

9.6 Angenommen, einem Vektorelement soll ein neuer Wert zugewiesen werden. Der verwendete Index ist aber unzulässig (d.h. negativ oder zu groß). Dann

- a) gibt der Compiler eine Fehlermeldung aus.
- b) wird das Programm bei der Ausführung der Anweisung mit einer Fehlermeldung beendet.
- ☒ c) gerät das Programm durch die Ausführung der Anweisung in einen undefinierten Zustand und kann abstürzen.

9.7 Nach der Vektordefinition

```
char a[] = { '0', 'K', '?' };
```

hat `a[2]` den Wert '?' (dezimal 63)

9.8 Für den String

```
char name[] = "Jim";
```

liefert `strlen(name)` den Wert 3 und `sizeof(name)` den Wert 4.

9.9 Welche der folgenden Definitionen sind korrekte String-Definitionen?

- ☒ a) `char s1[] = "Test!";`
- b) `char s2[5] = "Test!";`
- ☒ c) `char s3[10] = "Test!";`
- d) `char s4[] = { 'T', 'e', 's', 't', '!' };`
- ☒ e) `char s5[] = { 'T', 'e', 's', 't', '!', '\0' };`

9.10 Ein Vektor kann an einen anderen Vektor zugewiesen werden, wenn beide Vektoren gleich lang sind und ihre Elemente denselben Datentyp haben.

- ☐ Richtig
- ☒ Falsch

9.11 Zum Vergleich zweier Strings gibt es die Standardfunktion

- ☒ a) strcmp
- b) strlen
- c) strcat

9.12 Die Standardfunktion `strcpy()` kopiert einen String in einen char-Vektor. Beim Aufruf ist der char-Vektor als erstes Argument und der zu kopierende String als zweites Argument anzugeben. Ein Beispiel:

```
#include <string.h> // Prototyp von strcpy()
...
char msg[6];
strcpy( msg, "Heads up!");
```

Hier ist aber der Aufruf von `strcpy()` nicht korrekt, weil

- ☒ a) der Vektor `msg` zu klein ist. (Er muss mindestens die Größe 10 haben.)
- b) `msg` kein Vektor ist.
- c) weil `"Heads up!"` kein String ist.

9.13 Für einen beliebigen Vektor `v` und einen Index `i` sind die Ausdrücke `&v[i]` und `v+i` äquivalent.

- ☒ [X] Richtig
- ☐ [] Falsch

9.14 Gegeben sei der String

```
char fluss[24] = "fluss=Isar";
```

Dann repräsentiert der Ausdruck `fluss+6`

- ☒ a) die Adresse des Zeichens 'I'.
- b) das Zeichen '='.
- c) den String ab dem 6. Zeichen, also `"=Isar"`.

9.15 Gegeben sei der char-Vektor `word` mit der Länge 16. Die Anweisung, um das nächste Wort, aber höchstens 15 Zeichen des Wortes, von der Standardeingabe in den Vektor `word` einzulesen, lautet:

- ☒ a) `scanf("%15s", word);`
- b) `scanf("%s", word);`
- c) `printf("%s", word);`
- d) `scanf("%*s", word);`

9.16 Die zwei char-Vektoren `str1` und `str2` speichern jeweils einen String. Dann bewirkt der Ausdruck

```
str1 < str2
```

- a) eine Fehlermeldung des Compilers.
- b) den Vergleich der beiden Strings.
- ☒ c) den Vergleich zweier Adressen.

9.17 Die Variable `i` und der Vektor `arr` sind wie folgt definiert:

```
int i = 2;  
double arr[5] = { 15.7, 10.1, 21.4, 17.5, 13.9 };
```

Welche der folgenden Ausdrücke sind zulässig?

- ☒ a) `arr[2*i]`
- ☐ b) `arr[i+3]`
- ☐ c) `arr[i-2]`

9.18 Gegeben seien die Variable `i` und der Vektor `arr` aus der vorhergehenden Frage, sowie die Variable

```
double sum = 0.0;
```

Dann summiert die folgende `for`-Schleife alle Elemente des Vektors `arr`, so dass die Variable `sum` anschließend die Summe enthält:

- ☒ a) `for( i = 0; i < 5; ++i) sum += arr[i];`
- ☐ b) `for( i = 0; i < sizeof(arr); ++i) sum = sum + arr[i];`

9.19 Gegeben sei ein zweidimensionaler Vektor `matrix` mit Elementen vom Typ `int`. Dann ist `matrix[0]`

- ☐ a) das erste Element der Matrix vom Typ `int`.
- ☒ b) die erste Zeile der Matrix.
- ☐ c) die erste Spalte der Matrix.

9.20 `mat` ist eine Matrix mit Elementen vom Typ `double`, die aus 4 Zeilen und 3 Spalten besteht. Dann verdoppelt die folgende Anweisung das erste Element in der letzten Zeile:

- ☒ a) `mat[3][0] *= 2;`
- ☐ b) `mat[0][3] *= 2;`
- ☐ c) `mat[3][0] = mat[3][0] + mat[3][0];`

Funktionen

10.1 Wenn in einer Quelldatei die Funktion `main()` und andere Funktionen definiert werden, gilt für ihre Reihenfolge:

- ☒ a) Die Funktionen dürfen in beliebiger Reihenfolge definiert werden.
- ☐ b) Die Funktion `main()` muss zuerst definiert werden.
- ☐ c) Zuerst müssen alle anderen Funktionen definiert werden, dann die Funktion `main()`.

10.2 Die Parameter einer Funktion sind lokale Variablen, die beim Aufruf mit den Werten der Argumente initialisiert werden.

- ☒ Richtig
- ☐ Falsch

10.3 Eine Funktion, die keinen Wert zurückgibt, hat den Typ

- a) `int`
- ☒ b) `void`
- c) `char`
- d) `NULL`

10.4 Der Prototyp einer Funktion unterscheidet sich vom Funktionskopf der Definition durch

- a) nichts.
- ☒ b) ein Semikolon und evtl. fehlende oder andere Parameternamen.
- c) Default-Werte, die der Prototyp enthalten kann.

10.5 Jede Funktionsdefinition enthält mindestens eine `return`-Anweisung.

- ☐ Richtig
- ☒ Falsch

10.6 Die Definition einer Funktion kann mehrere `return`-Anweisungen enthalten.

- ☒ Richtig
- ☐ Falsch

10.7 Beim Aufruf einer Funktion muss ein Argument

- ☒ a) ein Ausdruck sein, dessen Wert in den Typ des entsprechenden Parameters konvertiert werden kann.
- b) ein Ausdruck sein, dessen Typ mit dem Typ des entsprechenden Parameters übereinstimmt.
- c) eine Variable vom Typ des entsprechenden Parameters sein.

10.8 Bei der Definition einer Funktion muss der Programmierer darauf achten, dass die Namen von lokalen Variablen nicht in Konflikt mit Namen in anderen Funktionen stehen.

- ☐ Richtig
- ☒ Falsch

10.9 Die ausführbare Datei eines C-Programms enthält den Maschinencode einer gewöhnlichen Funktion, die 3-mal aufgerufen wird, ein-mal.

10.10 In C ist es nicht möglich, dass eine Funktion eines ihrer Argumente verändert.

- ☒ Richtig
- ☐ Falsch

10.11 Wenn beim Aufruf einer Funktion nicht für jeden Parameter ein Argument angegeben ist, erhalten die übrigen Parameter den Default-Wert 0.

- ☐ Richtig
- ☒ Falsch

10.12 Der Maschinencode einer `inline`-Funktion wird

- a) vom Compiler an der Stelle des ersten Aufrufs eingefügt.
- ☒ b) vom Compiler an jeder Stelle eingefügt, an der sie aufgerufen wird.
- c) wie bei einer normalen Funktion vom Linker eingefügt.

10.13 Um eine `inline`-Funktion aufrufen zu können, genügt es, ihren Prototyp am Anfang der Quelldatei anzugeben.

☐ Richtig

☒ Falsch

10.14 Eine `inline`-Funktion wird in verschiedenen Quelldateien benötigt. In diesem Fall sollten Sie die `inline`-Funktion in einer Header-Datei definieren.

10.15 Die Funktion `myFunc()` besitzt folgenden Prototyp:

```
int myFunc( char str[] );
```

Dann kann die Funktion `myFunc()` jedes Zeichen eines als Argument übergebenen Strings ändern.

☒ Richtig

☐ Falsch

10.16 Angenommen, beim Aufruf einer Funktion wird als Argument der Name eines Vektors angegeben. Der entsprechende Parameter der Funktion enthält dann

a) eine lokale Kopie des Vektors.

b) das erste Vektorelement.

☒ c) die Anfangsadresse des Vektors.

10.17 Eine Funktion, die sich selbst aufruft, wird als rekursive Funktion bezeichnet.

10.18 Wenn eine Funktion sich selbst aufruft, werden die lokalen Variablen

☒ a) mit jedem Aufruf neu angelegt.

b) nur beim ersten Aufruf angelegt.

c) abhängig vom Compiler neu angelegt.

10.19 Beim Aufruf einer rekursiven Funktion muss der Programmierer darauf achten, dass zur Laufzeit genügend Platz auf dem Stack verfügbar ist.

10.20 Gegeben ist folgende Funktion:

```
long func( int n)
{
    if( n < 1)          return 0;
    else if( n == 1)    return 1;
    else                return func(n-2) + func(n-1);
}
```

Dann liefert der Aufruf `func(3)` den Wert

a) 0 + 1, also 1.

☒ b) 1 + 1, also 2.

c) 1 + 2, also 3.

Speicherklassen

11.1 Der Geltungsbereich und die Lebensdauer eines Objekts wird durch die Position der Definition im Quellcode und dem dabei verwendeten Speicherklassen-Spezifizierer festgelegt.

11.2 Um ein Programm einfach und übersichtlich zu halten, sollten möglichst alle Variablen global definiert werden.

☐ Richtig

☒ Falsch

11.3 Eine Variable ist global in einer anderen Quelldatei definiert. Für den Zugriff auf die Variable

a) ist keine weitere Aktion notwendig.

☒ **b)** muss die Variable mit einer extern-Deklaration importiert werden.

c) muss die Variable mit dem Spezifizierer extern definiert worden sein.

11.4 Die Deklaration, um die globale double-Variable `limit` in der aktuellen Quelldatei verfügbar zu machen, lautet:

☒ **a)** `extern double limit;`

b) `static double limit;`

c) `auto double limit;`

11.5 Um einer global definierten Variablen in einer neuen Quelldatei einen eigenen Anfangswert zu geben, kann die extern-Deklaration einen Initialisierer enthalten.

☐ Richtig

☒ Falsch

11.6 Ein innerhalb einer Funktion definiertes Objekt wird als lokal bezeichnet.

11.7 Das Code-Fragment

```
int n = 10;
void myFunc()
{
    int n = 20;
    printf("Wert von n = %d\n", n);
    ...
}
```

a) erzeugt eine Fehlermeldung des Compilers.

b) erzeugt die Ausgabe: Wert von n = 10.

☒ **c)** erzeugt die Ausgabe: Wert von n = 20.

11.8 Die Lebensdauer eines lokalen Objekts endet stets mit dem Verlassen des Blocks, in dem es definiert ist.

☐ Richtig

☒ Falsch

11.9 Der Anfangswert einer lokalen, nicht statischen Variablen, die ohne Initialisierung definiert wurde, ist

- ☒ a) undefiniert.
- ☐ b) 0.
- ☐ c) abhängig vom Compiler.

11.10 Um zu erreichen, dass eine lokale Variable beim Wiedereintritt in den Block, in dem sie definiert ist, noch mit ihrem alten Wert vorhanden ist, muss sie als static - Variable definiert werden.

11.11 Zwei nicht lokale, als `static` definierte Variablen dürfen denselben Namen haben, wenn sie in verschiedenen Quelldateien definiert sind.

- ☒ [X] Richtig
- ☐ [] Falsch

11.12 Nicht initialisierte statische Variablen haben den Anfangswert 0.

11.13 Eine `static`-Variable wird innerhalb eines Funktionsblocks definiert und initialisiert. Dann erfolgt die Initialisierung der Variablen

- ☒ a) beim Laden des Programms.
- ☐ b) beim 1. Eintritt in den Block.
- ☐ c) bei jedem Eintritt in den Block.

11.14 Die Lebensdauer statischer Objekte ist wie die Lebensdauer globaler Objekte die gesamte Ausführungszeit des Programms.

- ☒ [X] Richtig
- ☐ [] Falsch

11.15 Gegeben sei folgende Funktion:

```
long func()  
{  
    static long a = 1;  
    return a *= 2;  
}
```

Dann gibt der dritte Aufruf der Funktion den Wert 8 zurück.

11.16 Ein Programm verwaltet in einer Quelldatei die zentrale Datenbasis des Programms. Auf diese Daten soll in anderen Quelldateien nur unter der Kontrolle von Funktionen zugegriffen werden können, die in derselben Quelldatei definiert sind. Dies wird erreicht, indem die Daten als static definiert werden.

11.17 Eine Funktion kann nicht innerhalb einer anderen Funktion definiert werden.

- ☒ [X] Richtig
- ☐ [] Falsch

11.18 Einige global definierte Objekte und Funktionen werden in verschiedenen Quelldateien des Programms benötigt. In diesem Fall sollten Sie diese Objekte und Funktion in einer Header-Datei deklarieren.

11.19 Um eine `static`-Funktion aufzurufen, die in einer anderen Quelldatei definiert ist, genügt es, den Prototyp der Funktion anzugeben.

☐ Richtig

☒ Falsch

11.20 Für die Funktionen einer Quelldatei soll eine Hilfsfunktion `calc()` definiert werden, die in anderen Quelldateien nicht aufrufbar ist. Die Funktion besitzt zwei Parameter `x` und `y` vom Typ `double` und gibt einen `int`-Wert zurück. Dann lautet der Funktionskopf der Funktion:

☒ a) `static int calc( double x, double y)`

☐ b) `int calc( double x, double y)`

☐ c) `static int calc( double x, y)`

☐ d) `int calc( double x, y)`

Bitoperatoren

12.1 Die Bitoperatoren sind nur auf Operanden mit einem ganzzahligen Datentyp anwendbar.

☒ Richtig

☐ Falsch

12.2 Der Ausdruck `3 | 6` hat den dezimalen Wert

☐ a) 6.

☒ b) 7.

☐ c) 9.

12.3 Der Wert des Ausdrucks `7 & 9` hat das Bitmuster 00000001.

12.4 Das Code-Fragment

```
int a = 5, b = 6;  
printf(" %d", a^b);
```

erzeugt die Ausgabe 3.

12.5 Gegeben seien zwei beliebige `short`-Variablen `a` und `b`. Dann ergibt der Ausdruck $(a^b)^a$ den Wert

☐ a).

☒ b).

☐ c) a^b .

12.6 Für zwei `int`-Variablen `x` und `y` liefert der Ausdruck $(x | y) \& y$ wieder den Wert von `x`.

☐ Richtig

☒ Falsch

12.7 Die Anweisung

```
printf("%d %d\n", 1&2, 1&&2 );
```

erzeugt die Ausgabe

- a) 0 0.
- ☒ b) 0 1.
- c) 1 0.
- d) 1 1.

12.8 Um durch die Anweisung

```
x |= MASK;
```

in x das 0. und 2. Bit zu setzen, muss MASK welchen Wert haben?

- ☒ a) 5
- b) 3
- c) 9

12.9 Die Operatoren ~ (bitweises NICHT) und ! (logisches NICHT) haben den selben Vorrang.

- ☒ [X] Richtig
- ☐ [] Falsch

12.10 Für eine beliebige int-Variable x hat der Ausdruck x&~x den Wert 0.

12.11 Gegeben seien zwei int-Variablen a und b. Dann bewirkt die Anweisung

```
a &= ~b;
```

- ☒ a) die Löschung der Bits in a, die in b gesetzt sind.
- b) die Löschung der Bits in a, die in b gelöscht sind.
- c) die Invertierung der Bits in b und anschließende Zuweisung an a.

12.12 Die Anweisung(en), um das 0. und 2. Bit einer int-Variable x zu löschen, lautet

- ☒ a) a &= -5;
- b) a ^= -5;
- c) a |= -5;

12.13 Die folgende Schleife

```
for( int n = 0; n <= 5; ++n) printf(" %d", 1<<n);
```

erzeugt die Ausgabe

- ☒ a) 1 2 4 8 16 32
- b) 1 2 3 4 5 6
- c) 2 4 8 16 32 64

12.14 Der Ausdruck $5 \ll 3$ hat den dezimalen Wert

- a) 8.
- b) 15.
- ☒ c) 40.

12.15 Der Ausdruck $18 \gg 2$ ist gleichbedeutend mit $18 / 4$. Sein Wert ist daher 4.

- ☒ [X] Richtig
- ☐ [] Falsch

12.16 Gegeben sei eine `short`-Variable `status` und eine ganzzahlige Variable `i` mit einem Wert zwischen 0 und 15. Dann wird das `i`-te Bit von `status` durch die Anweisung

`status ^= 1 << i;`

- a) gelöscht.
- b) gesetzt.
- ☒ c) invertiert.
- d) nicht verändert.

12.17 Für eine ganzzahlige Variable `m` liefert der Ausdruck $(m \gg 5) \& 1$ den Wert des

- ☒ a) 5-ten Bits.
- b) 7-ten Bits.
- c) 1-ten Bits.

12.18 Der Ausdruck, der den Wert der Bitgruppe 3. bis 5. Bit der `short`-Variablen `access` liefert, lautet

- ☒ a) `(access >> 3) & 7`
- b) `(access >> 7) & 3`
- c) `access & 7`
- d) `(access & 7) >> 3`

12.19 Gegeben sei eine ganzzahlige Variable `n`. Dann liefert der Ausdruck $(n \gg 4) \ll 4$ das gleiche Ergebnis wie `n & ~0xF`.

- ☒ [X] Richtig
- ☐ [] Falsch

12.20 Gegeben sei eine Variable `n` vom Typ `unsigned int`. Nach Ausführung der folgenden Schleife

```
unsigned int s = 0, k;  
for( k = n; k != 0; k >>= 1) s += (k & 1);
```

enthält die Variable `s`

- ☒ a) die Anzahl der in `n` gesetzten Bits.
- b) die Anzahl der in `n` gelöschten Bits.
- c) die Anzahl der Schleifendurchläufe.

Zeiger

13.1 Ein Zeiger repräsentiert die Adresse und den Typ eines Objekts.

13.2 Für eine Variable `var` ist der Ausdruck `&var` ein Zeiger auf `var`.

☒ Richtig

☐ Falsch

13.3 Die Zeiger `p1` und `p2` haben denselben Typ. Die Zuweisung, um `p2` auf dasselbe Objekt zeigen zu lassen, auf das `p1` zeigt, lautet

☒ a) `p2 = p1;`

☐ b) `p2 = &p1;`

☐ c) `p2 = *p1;`

13.4 Wenn `a` eine `int`-Variable ist, hat der Ausdruck `&a` den Typ

☒ a) `int *`

☐ b) `int`

☐ c) `unsigned int`

☐ d) `void *`

13.5 Gegeben sei eine `double`-Variable `x`. Die Definition eines Zeigers `ptr`, der so initialisiert wird, dass er auf `x` zeigt, lautet

☒ a) `double *ptr = &x;`

☐ b) `double *ptr = x;`

☐ c) `int *ptr = &x;`

13.6 Die Adresse einer Zeigervariablen `ptr` liefert der Ausdruck

☐ a) `ptr.`

☐ b) `*ptr.`

☒ c) `&ptr.`

13.7 Für einen `char`-Zeiger `cPtr` und einen `float`-Zeiger `fPtr` ist `sizeof(cPtr)` kleiner als `sizeof(fPtr)`.

☐ Richtig

☒ Falsch

13.8 C garantiert, dass jede gültige Adresse von 0 or NULL verschieden ist.

13.9 Gegeben sei der Zeiger `p`, der auf eine `int`-Variable `i` zeigt. Dann bewirkt der Ausdruck `2.5**p`

☐ a) eine Fehlermeldung des Compilers.

☒ b) die Multiplikation der `int`-Variablen `i` mit `2.5`.

☐ c) die Potenzierung von `2.5` mit dem Wert von `p`.

13.10 Der `float`-Zeiger `ptr` enthalte die Adresse 10000. Nach der Zuweisung

```
ptr = ptr + 1;
```

enthält `ptr` die Adresse 10004 // 10000 + sizeof(float).

13.11 Seien `arr` ein Vektor und `i` ein Index. Dann liefern die beiden Ausdrücke `arr+i` und `&arr[i]` den gleichen Zeiger.

☒ Richtig

☐ Falsch

13.12 Da der Vektorname `arr` ein Zeiger auf das Element `arr[0]` ist, zeigt `arr` nach der Anweisung `++arr;` auf `arr[1]`.

☐ Richtig

☒ Falsch (`arr` ist ein konstanter Zeiger.)

13.13 Gegeben seien folgende Definitionen:

```
int a[] = { 10, 20, 30, 40, 50}, sum = 0, *p;
```

Dann durchläuft der Zeiger `p` in der folgenden Schleife den Vektor `a`, um die Summe der Vektorelemente zu bilden:

☒ a) `for(p = a; p < a+5; ++p) sum += *p;`

☐ b) `for(p = a; p < a+5; p += 1) sum += p;`

☐ c) `for(p = &a; p < &a[5]; ++p) sum += *p;`

13.14 Die `int`-Zeiger `p1` und `p2` zeigen auf die `int`-Variablen `a1` und `a2`. Dann wird durch folgende Anweisung

```
printf("%d\n", *(p1+p2) );
```

☒ a) der Compiler eine Fehlermeldung ausgeben. (Zeiger können nicht addiert werden.)

☐ b) zur Laufzeit ein undefinierter Wert angezeigt.

☐ c) der Wert der Summe `*p1+*p2`, also `a1+a2`, angezeigt.

13.15 Nach den Definitionen

```
short a[5], *p = &a[2];
```

hat der Ausdruck `p - a` den Wert 2.

13.16 Die Funktion `calc()` erhält als Argumente einen `int`-Wert und zwei `double`-Zeiger. Sie gibt keinen Wert zurück. Dann lautet der Prototyp der Funktion:

☒ a) `void calc( int, double *, double *);`

☐ b) `void calc( int, double, double);`

☐ c) `int calc( int *, double *, double *);`

13.17 Sei `var` eine beliebige Variable. Dann liefert der Ausdruck

```
*(unsigned char *)&var
```

das niederwertigste Byte (Low Byte) von `var`.

☒ Richtig

☐ Falsch

13.18 Nach den Definitionen

```
void mul(int *ptr, int i) { *ptr *= i; }  
int a = 10, b = 20, *p = &a;
```

können Sie die Funktion `mul()` wie folgt aufrufen:

- ☒ a) `mul(&a, 2);`
- ☐ b) `mul(&b, *p);`
- ☐ c) `mul(p, *p+5);`

13.19 Die Funktionsdefinition

```
int *min(int a, int b) { return a <= b ? &a : &b; }
```

ist korrekt und liefert einen Zeiger auf das kleinere Argument.

- ☐ Richtig
- ☒ Falsch (a und b sind temporäre, lokale Variablen.)

13.20 Die Funktion `func()` besitzt folgenden Prototyp:

```
char *func(void);
```

Dann bewirkt die Anweisung

```
*func() = 'X';
```

- a) eine Fehlermeldung des Compilers.
- b) die Zuweisung eines neuen Return-Wertes an die Funktion.
- ☒ c) die Zuweisung eines Wertes an das Objekt, auf das der Return-Wert zeigt.

Dynamische Speicherplatzverwaltung

14.1 Zur dynamischen Reservierung eines neuen Speicherblocks stehen in C die Funktionen `malloc` und `calloc` zur Verfügung.

14.2 Ein dynamisch reservierter Speicherblock stellt stets einen lückenlosen, zusammenhängenden Speicherbereich dar.

- ☒ Richtig
- ☐ Falsch

14.3 Wenn `iPtr` ein `int`-Zeiger ist, enthält `iPtr` nach der Anweisung

```
iPtr = (int *)malloc(5);
```

den Wert 0 oder die Adresse eines Speicherblocks für 5 `int`-Werte.

- ☒ Richtig
- ☐ Falsch Richtig ist: `malloc( 5*sizeof(int))`

14.4 Gegeben seien ein `float`-Zeiger `fPtr` und eine `int`-Variable `n`. Dann reserviert folgende Anweisung Speicherplatz für einen Vektor mit `n float`-Elementen (ohne Prüfung des Return-Wertes):

- ☒ a) `fPtr = (float *)malloc(n*sizeof(float));`
- ☐ b) `fPtr = malloc(n*sizeof(float));`
- ☐ c) `fPtr = calloc(n, sizeof(float));`

14.5 Sei `lPtr` ein `long`-Zeiger. Dann reserviert die folgende Anweisung Speicher für einen Vektor mit 100 `long`-Elementen und initialisiert alle Elemente mit 0, sofern genügend Speicher zur Verfügung steht:

- ☐ a) `lPtr = calloc( 100, sizeof(long));`
- ☐ b) `lPtr = malloc( 100 * sizeof(long));`
- ☐ c) `free(lPtr);`

14.6 Der Zeiger `dPtr` zeige auf das erste Element eines dynamisch erzeugten Vektors mit 100 `double`-Elementen. Dann wird mit

```
dPtr[1] = 2.2;
```

dem zweiten Element der Wert 2.2 zugewiesen.

- ☒ Richtig
- ☐ Falsch

14.7 Wenn der Zeiger `ptr` auf den Anfang eines dynamisch reservierten Speicherblocks zeigt, liefert der Ausdruck `sizeof(*ptr)` die Größe des Speicherblocks in Anzahl Bytes.

- ☐ Richtig
- ☒ Falsch

14.8 Gegeben sei folgender dynamisch erzeugter `int`-Vektor:

```
int *p = malloc( 10*sizeof(int));
```

Dann ist das erste Element des Vektors

- a) `p`.
- ☒ b) `*p`.
- ☐ c) `p[0]`.

14.9 Die Anweisung, um den dynamisch reservierten Speicherblock freizugeben, auf den der Zeiger `p` zeigt, lautet `free(p)`.

14.10 Im Anschluss an die Definitionen

```
int arr[10], *iPtr = arr;
```

bewirkt die Anweisung `free(iPtr);`

- a) die Freigabe des durch den Vektor `arr` belegten Speichers.
- b) eine Fehlermeldung des Compilers.
- ☒ c) eine Fehlermeldung zur Laufzeit des Programms.

14.11 Gegeben sei ein `int`-Zeiger `iPtr`, der auf das erste Element eines dynamisch erzeugten Vektors mit 10 `int`-Elementen zeigt. Dann wird mit

```
free(iPtr+5);
```

der Speicher des Vektors ab dem Element mit dem Index 5 freigegeben.

☐ Richtig

☒ Falsch

14.12 Wenn für einen dynamisch reservierten Speicherblock nicht die Funktion `free()` aufgerufen wird, so wird der Speicherblock

☒ a) freigegeben, wenn das Programm beendet wird.

☐ b) automatisch freigegeben, wenn kein Zeiger mehr auf ihn verweist.

☐ c) nie freigegeben.

14.13 Nach dem Aufruf der Funktion `free()` kann nicht überprüft werden, ob der dynamisch reservierte Speicher erfolgreich freigegeben wurde.

☒ Richtig

☐ Falsch

14.14 Die folgende Zeile definiert einen Zeiger `mp` auf einen Vektor mit 10 Elementen vom Typ `int`:

```
int (*mp)[10];
```

Dann reserviert die folgende Anweisung dynamisch Speicher für eine 5 x 10- Matrix (= zweidimensionaler Vektor mit 5 Zeilen und 10 Spalten) mit `int`-Elementen und `mp` zeigt auf die erste Zeile der Matrix:

☒ a) `mp = malloc(5*10*sizeof(int));`

☐ b) `mp = malloc(5*10);`

☐ c) `mp = malloc(5*10*sizeof(*mp));`

14.15 Sei `mp` ein Zeiger, der auf die erste Zeile einer dynamisch reservierten 5 x 10-Matrix mit `int`-Elementen zeigt. Dann wird durch

```
mp[4][0] = 100;
```

dem ersten Element in der letzten Zeile der Wert 100 zugewiesen.

☒ Richtig

☐ Falsch

14.16 Mit der Funktion `realloc` or `realloc()` kann die Größe eines dynamisch reservierten Speicherblocks verkleinert oder vergrößert werden.

14.17 Gegeben seien zwei `float`-Zeiger `fp1` und `fp2`, wobei `fp1` auf einen dynamischen Speicherblock für 1000 `float`-Werte zeigt. Die Anweisung, um den Speicherblock zu verdoppeln, so dass `fp2` auf den neuen Block zeigt, lautet (ohne Prüfung des Return-Wertes):

☒ a) `fp2 = realloc( fp1, 2*1000*sizeof(float));`

☐ b) `fp2 = calloc( fp1, 2*1000*sizeof(float));`

☐ c) `fp2 = malloc( fp1, 2*1000*sizeof(float));`

14.18 Bei der Vergrößerung eines dynamisch reservierten Speicherblocks mit `realloc()` kann es notwendig werden, den Speicherblock zu verschieben. In einem solchen Fall werden die Daten des alten Blocks automatisch in den neuen Block kopiert.

☒ Richtig

☐ Falsch

14.19 Beim Aufruf von `realloc()` kann es vorkommen, dass der angeforderte Speicherplatz nicht zur Verfügung steht. Daher muss der Programmierer vor dem Aufruf die Daten des alten Blocks sichern.

☐ Richtig

☒ Falsch

14.20 Die Funktion `func()` gibt einen Zeiger auf einen dynamisch reservierten Speicherblock zurück. Für die Freigabe des Speichers gilt dann:

a) Nur die Funktion selbst kann den Speicher wieder freigeben.

☒ **b)** Der Aufrufer der Funktion sollte den Speicher freigeben, wenn er ihn nicht mehr benötigt.

c) Der Speicher bleibt zwangsläufig bis zur Beendigung des Programms reserviert.

Strukturierte Datentypen

15.1 Der Aufbau eines Datensatzes mit den Namen und Typen der Datenfelder wird in C durch die Definition

☒ **a)** einer Struktur durchgeführt.

b) eines Vektors durchgeführt.

c) einer Datei durchgeführt.

15.2 Eine mögliche Definition für einen Datensatz, der nur den Namen und Preis eines Artikels enthält, lautet:

```
struct Artikel { char namen[64]; double preis; };
```

☒ Richtig

☐ Falsch

15.3 Die Elemente verschiedener Strukturen dürfen den gleichen Namen haben.

☒ Richtig

☐ Falsch

15.4 Durch die Definition eines Strukturtyps `struct Time` mit den Elementen `hour`, `min` und `sec` wird im Programm folgende Anzahl Byte belegt:

☒ **a)** 0

b) `sizeof(struct Time)`

c) `sizeof(hour) + sizeof(min) + sizeof(sec)`

15.5 Ein Strukturtyp wird in verschiedenen Quelldateien benötigt. In diesem Fall sollten Sie die Struktur in einer Header-Datei definieren.

15.6 Wenn `struct Time` ein Strukturtyp ist, der ein Element `hour` enthält, ist `Time.hour` das Element `hour`.

☐ Richtig

☒ Falsch

15.7 Speicher von der Größe der Struktur wird reserviert, wenn

a) der Strukturtyp definiert wird.

b) die Header-Datei mit der Strukturdefinition inkludiert wird.

☒ **c)** eine Variable vom Typ der Struktur angelegt wird.

15.8 Gegeben sei folgende Definition:

```
struct Cylinder { double radius, height; };
```

Ein Objekt `can1` vom Typ dieser Struktur kann wie folgt definiert und mit 0,5 und 10 initialisiert werden:

☒ **a)** `struct Cylinder can1 = { 0.5, 10.0 };`

b) `Cylinder can1 = { 0.5, 10.0 };`

15.9 Eine Strukturvariable kann an eine andere, nicht konstante Strukturvariable gleichen Typs zugewiesen werden.

☒ Richtig

☐ Falsch

15.10 Der Speicher für einer Strukturvariable wird stets so belegt, dass zwischen den Elementen keine Lücken entstehen.

☐ Richtig

☒ Falsch

15.11 Bei der Zuweisung von Strukturvariablen gleichen Typs

a) erzeugt der Compiler eine Fehlermeldung.

☒ **b)** wird der Speicherbereich der Quellvariablen Byte für Byte in den Speicherbereich der Zielvariablen kopiert.

c) wird jedes Element der Quellvariablen in das entsprechende Element der Zielvariablen kopiert.

15.12 Der linke Operand des Punkt-Operators muss eine Strukturvariable (ein Objekt vom Typ einer Struktur) sein.

15.13 Der Strukturtyp `Cylinder_t` sei wie folgt definiert:

```
typedef Cylinder_t struct Cylinder;
```

Die Definition einer Variablen vom Typ `Cylinder_t` und eines Zeigers darauf lautet

```
Cylinder_t can1, *canPtr = &can1;
```

☒ Richtig

☐ Falsch

15.14 Der Zeiger `pTm` zeige auf ein Objekt vom Typ `struct Time`. Dann repräsentiert der folgende Ausdruck das Element `sec` des Objekts:

- ☒ a) `(*pTm).sec`
- ☐ b) `pTm->sec`
- ☐ c) `pTm.sec`.

15.15 Gegeben seien die Definitionen

```
struct X { int count; /* Weitere Elemente. */ };  
int incX( struct X *px) { return ++px->count; }
```

Dann inkrementiert die Funktion `incX()` den Zeiger `px` und liefert den Wert `count` der nachfolgenden Struktur.

- ☐ Richtig
- ☒ Falsch

15.16 Gegeben sei ein Zeiger `px` auf `struct X`. Die Anweisung, um dynamisch ein Objekt vom Typ `struct X` zu erzeugen und `px` darauf zeigen zu lassen, lautet:

```
px = malloc( sizeof(struct X));
```

- ☒ Richtig
- ☐ Falsch

15.17 Die Größe eines Objekts vom Typ einer Union ist gleich

- a) der Größe des ersten Elements.
- b) der Summe der Größen aller Elemente.
- ☒ c) der Größe des größten Elements.

15.18 Die Definition einer Union `Word` mit einem Element vom Typ `short` und einem Element, das ein `char`-Vektor mit zwei Elementen ist, lautet:

```
union Word { short w; char b[2]; };
```

15.19 Gegeben sei folgende Definition:

```
union LongFloat{ long n; float x; } var;  
var.x = 1;
```

Dann hat auch `var.n` den Wert 1.

- ☐ Richtig
- ☒ Falsch

15.20 Mehrere Bitfelder, die in ein Rechnerwort der Größe `sizeof(int)` passen, werden so ausgerichtet, dass ein nachfolgendes Bitfeld lückenlos an das vorhergehende Bitfeld anschließt.

- ☒ Richtig
- ☐ Falsch

High-Level-Dateizugriff

16.1 Die Standardfunktionen für die Byte-orientierte Ein-/Ausgabe sind in welcher Header-Datei deklariert?

- ☒ a) `stdio.h`
- ☐ b) `stdlib.h`
- ☐ c) `string.h`

16.2 Jede geöffnete Datei ist mit einem Stream verbunden, der für die Datenübertragung zuständig ist.

16.3 Eine Datei ist aus der Sicht eines C-Programms eine Folge von

- ☒ a) Bytes.
- ☐ b) Zeilen.
- ☐ c) Datensätzen.

16.4 Die Funktion `fopen()` liefert als Return-Wert immer den Null-Zeiger, wenn die zu öffnende Datei nicht existiert.

- ☐ ☐ Richtig
- ☒ ☐ Falsch

16.5 Mit dem Zugriffsmodus `"wb"` wird eine Datei zum Schreiben im Binärmodus geöffnet.

16.6 Beim Öffnen einer bestehenden Datei bleibt der Inhalt der Datei stets erhalten.

- ☐ ☐ Richtig
- ☒ ☐ Falsch

16.7 Der Zugriffsmodus, um eine Datei zum Lesen und Schreiben zu öffnen (engl. *open for update*), lautet

- ☒ a) `"r+"`
- ☐ b) `"rb+"`
- ☐ c) `"w+"`
- ☐ d) `"wb+"`

16.8 Eine Textdatei, die im Modus `"r+"` geöffnet wird,

- a) wird neu erzeugt, falls sie noch nicht vorhanden ist.
- b) wird auf die Länge 0 gekürzt, falls sie vorhanden ist.
- ☒ c) muss bereits vorhanden sein.

16.9 Sei `fp` ein Zeiger vom Typ `FILE*`. Dann lautet die Anweisung, um die Textdatei `"Memo.txt"` zum Lesen und zum Schreiben ab Dateiende zu öffnen und `fp` auf den neuen Stream zeigen zu lassen:

- ☒ a) `fp = fopen("Memo.txt", "a+");`
- ☐ b) `fp = fopen("Memo.txt", "w+");`
- ☐ c) `fp = fopen("Memo.txt", "r+");`

16.10 Ein regulär ablaufendes Programm schließt am Ende alle geöffneten Dateien.

- ☒ ☐ Richtig
- ☐ ☐ Falsch

16.11 Die folgende Anweisung bewirkt, dass nach dem Schreiben in eine mit dem FILE-Pointer `fp` verbundene Datei die Daten vom Ausgabepuffer in die Datei übertragen werden.

- ☒ a) `fflush(fp);`
- b) `fwrite(fp, ...);`
- c) `fread(fp, ...);`

16.12 Wenn beim Öffnen einer bestehenden Datei kein Pfad angegeben ist, muss sich die Datei im folgenden Verzeichnis befinden:

- a) im Homeverzeichnis des Anwenders.
- ☒ b) im aktuellen Verzeichnis.
- c) im Verzeichnis, in dem sich das ausgeführte Programm befindet.

16.13 Gegeben sei folgendes Code-Fragment:

```
FILE * fp;  
char zeile[100];  
if( (fp = fopen("sprueche.txt", "r")) == NULL)  
    exit(1);
```

Wenn die Datei erfolgreich geöffnet wurde, kann die erste Zeile der Datei wie folgt in den Vektor `zeile` eingelesen werden:

```
if( fgets(zeile, sizeof(zeile), fp ) != NULL)  
{ /* Zeile verarbeiten. */ }
```

- ☒ Richtig
- ☐ Falsch

16.14 Wenn eine Datei zum Lesen und Schreiben geöffnet wurde, kann beliebig zwischen Lesen und Schreiben gewechselt werden.

- ☐ Richtig
- ☒ Falsch

16.15 Die Datei `personen` wurde mit dem FILE-Pointer `fp` zum Schreiben am Dateiende im Binärmodus geöffnet. Sie enthält Datensätze vom Typ `struct Person`. Die Anweisung, um einen weiteren Datensatz, der sich in der Variablen `aPerson` befindet, an die Datei anzuhängen, lautet:

```
if( fwrite( &aPerson, sizeof(aPerson), 1, fp) < 1)  
{ /* Fehlerbehandlung. */ } .
```

- ☒ Richtig
- ☐ Falsch

16.16 Für die Dateiverarbeitung im Binärmodus gilt:

- a) Textdateien dürfen nicht im Binärmodus geöffnet werden.
- b) Beim Lesen und Schreiben im Binärmodus werden andere Steuerzeichen interpretiert als im Textmodus.
- ☒ c) Im Binärmodus werden alle Bytes unverändert übertragen.

16.17 Die Möglichkeiten zur Formatierung von Daten, die Sie mit `printf()` nutzen können, stehen mit `fprintf()` auch für die Ausgabe in eine Datei zur Verfügung.

☒ Richtig

☐ Falsch

16.18 Eine Datei, die mit dem FILE-Pointer `fp` geöffnet wurde, enthalte Datensätze vom Typ `struct Record`. Nach dem Lesen des ersten Datensatzes liefert dann der Funktionsaufruf `ftell(fp)` den Wert:

☒ `sizeof(struct Record)`

☐ `_countof(struct Record)`

☐ undefiniert

☐ 1

16.19 Die Anweisung, um die aktuelle Position in einer mit dem FILE-Pointer `fp` verbundenen Datei auf das Dateiende zu setzen, lautet:

☒ `fseek(fp, 0L, SEEK_END);`

☐ `fseek(fp, EOF, 0);`

☐ `fsetpos(fp, EOF);`

16.20 Auf Systemen, die zwischen Binärmodus und Textmodus unterscheiden, sollten die Funktionen `fread()` und `fwrite()` zum Lesen und Schreiben von Datensätzen nur auf Dateien angewendet werden, die im Binärmodus geöffnet wurden.

☒ Richtig

☐ Falsch